Embedded Module Architecture 実装ガイド

リポジトリの各モジュールで学ぶ設計パターンの実践

バージョン 1.2

作成日 2026 年 3 月 18 日

著者 Shiozawa

本ドキュメントは、Embedded Module Architecture リポジトリに
実装されている各モジュールを題材に、
設計パターンの適用方法を具体的なコードで解説するガイドです。

設計仕様書（設計ルール）・教科書（パターンの学習）と併せてお読みください。

改訂履歴

版	日付	著者	変更内容
1.0	2026-03-15	Shiozawa	初版作成。全 14 モジュールの実装解説
1.1	2026-03-17	Shiozawa	関連ドキュメントの相互参照を追加。日付・Rev 番号を全ドキュメントと同期
1.2	2026-03-18	Shiozawa	ESP32 固有の記述を Arduino/組み込み汎用に変更。ADC コメント・IRAM_ATTR 注記等を汎用化

目次

改訂履歴 1

1 この資料について 4
- 1.1 位置づけ . . . 4
- 1.2 読み方 . . . 4
- 1.3 モジュール一覧と学習レベル . . . 4

2 【基本】LedModule — 最小構成の基本パターン 5
- 2.1 ヘッダー（LedModule.h） . . . 5
- 2.2 実装（LedModule.cpp） . . . 6
- 2.3 ProjectConfig.h での定義 . . . 6

3 【基本】ButtonModule — デバウンスパターン 7
- 3.1 Config/Data 構造体 . . . 7
- 3.2 デバウンスの実装 . . . 7
- 3.3 ロジックフェーズでの使用例 . . . 8

4 【基本】BatteryModule — ADC + 移動平均フィルタ 8
- 4.1 Config/Data 構造体 . . . 9
- 4.2 リングバッファと移動平均 . . . 9

5 【基本】ServoModule — PWM 出力パターン 10
- 5.1 Config/Data 構造体 . . . 10
- 5.2 updateOutput() の実装 . . . 11

6 【基本】BuzzerModule — deinit() パターン 11
- 6.1 Data 構造体のリクエストパターン . . . 11
- 6.2 updateOutput() の実装 . . . 12
- 6.3 deinit() の実装 . . . 12

7 【中級】Mpu6500Module — I2C バスパターン 13
- 7.1 バスポインタの受け取り . . . 13
- 7.2 main.cpp でのバス管理 . . . 14

8 【中級】DisplayBoardModule — 入出力統合 + ライブラリ内部管理 14
- 8.1 なぜ 1 モジュールに統合するのか . . . 14
- 8.2 ライブラリの内部管理パターン . . . 15
- 8.3 Data 構造体の入出力統合 . . . 15
- 8.4 ロジックフェーズでの表示内容設定 . . . 16

9 【中級】CameraModule — PSRAM バッファ + フレーム管理 16
- 9.1 Config の大量ピン管理 . . . 17
- 9.2 フレームバッファの管理 . . . 17

10 【応用】DriveMotorModule — PWM 2 ピン制御 + 統合部品 18
10.1 2 つの使い方 18
10.2 逆転保護 19

11 【応用】ChassisModule — 機能統合パターン 19
11.1 内部モジュールの保持 19
11.2 親モジュールからの制御委譲 20

12 【応用】BleModule — スレッドセーフティパターン 21
12.1 内部バッファと volatile フラグ 21
12.2 コールバック→内部バッファ→ SystemData 22
12.3 BLE コールバックの登録 23
12.4 送信の流れ（ロジックフェーズ→ updateOutput()） 23

13 【応用】WifiModule — ステートマシンパターン 24
13.1 状態定義 24
13.2 ステートマシンの実装 24
13.3 ロジックフェーズでの使用例 25

14 【応用】EncoderModule — 割り込み連携パターン 26
14.1 割り込みハンドラの設計 26
14.2 ISR → volatile → updateInput() の流れ 27
14.3 deinit() で割り込みを解除 28

1 この資料について

1.1 位置づけ

本リポジトリには 2 つのプロジェクト（`sample/`: サンプル実装、`verified/`: 動作確認済み）と 3 つのドキュメントがある。

ドキュメント	ディレクトリ	内容
設計仕様書	`doc/`	アーキテクチャのルール・API 仕様（「何を守るか」）
教科書	`doc-textbook/`	パターンの段階的な学習（「なぜそうするか」）
実装ガイド（本書）	`doc-guide/`	実コードの解説（「実際にどう書いたか」）

！ サンプルコードについて

`sample/` に実装されているモジュール群のコードは、あくまで**アーキテクチャの設計パターンを具体的に示すためのサンプル実装**である。実機での動作検証は行っていないため、実プロジェクトへの適用時は十分なテストを実施すること。動作確認済みのコードは `verified/` を参照すること。

1.2 読み方

本書では、リポジトリに実装されている各モジュールを取り上げ、以下の構成で解説する。

1. **概要** — モジュールの役割と、そのモジュールで学べるパターン
2. **コード解説** — 実際のコードを引用し、設計ルールとの対応を示す
3. **ポイント** — 特に注目すべき設計判断やテクニック

モジュールは簡単なものから順に並べている。前半のモジュールで基本パターンを押さえたうえで、後半の応用パターンに進む構成になっている。

1.3 モジュール一覧と学習レベル

各モジュールには難易度レベルを付けている。まず【基本】を全て読んでパターンを身につけてから、必要に応じて【中級】【応用】に進むことを推奨する。

モジュール	種別	レベル	主な学習パターン
LedModule	出力	【基本】	最小構成の基本パターン
ButtonModule	入力	【基本】	デバウンス（ModuleTimer 活用）
BatteryModule	入力	【基本】	ADC 読み取り ＋ 移動平均フィルタ
ServoModule	出力	【基本】	PWM 制御（LEDC）
BuzzerModule	出力	【基本】	deinit() によるリソース解放
Mpu6500Module	入力	【中級】	I2C バスパターン ＋ レジスタ直接アクセス
DisplayBoardModule	入出力	【中級】	入出力統合 ＋ ライブラリ内部管理
CameraModule	入力	【中級】	PSRAM バッファ ＋ フレーム管理
DriveMotorModule	出力	【応用】	PWM 2 ピン制御 ＋ 統合モジュール部品
ChassisModule	出力	【応用】	機能統合系（内部にモジュールを持つ）
BleModule	入出力	【応用】	スレッドセーフティ（volatile フラグ方式）

モジュール	種別	レベル	主な学習パターン
WifiModule	入出力	【応用】	ステートマシン内蔵
EncoderModule	入力	【応用】	ハードウェア割り込み連携

▷ レベルの目安

- **【基本】** — 教科書の内容だけで理解できる。最初に全て読むべき
- **【中級】** — I2C/SPI 等の通信バスパターンが加わる。必要に応じて読む
- **【応用】** — 統合パターン・スレッドセーフティ・割り込みなど高度な内容。チャレンジ向け

2 【基本】LedModule — 最小構成の基本パターン

LedModule で学べるパターン

- Config 構造体によるピンアサインの外部注入
- Data 構造体のデフォルト値
- .h/.cpp 分離（循環依存回避）
- 出力モジュールの基本形（SystemData の値に従って動作するだけ）

2.1 ヘッダー（LedModule.h）

```cpp
#pragma once
#include <Arduino.h>
#include "IModule.h"

struct LedConfig {
    uint8_t ledPin;  // LED出力ピン
};

struct LedData {
    bool state = false;        // LED状態（デフォルト：消灯）
    uint8_t brightness = 0;    // 輝度（将来拡張用）
};

class LedModule : public IModule {
private:
    LedConfig _config;
public:
    LedModule(const LedConfig& config);
    bool init() override;
    void updateOutput(SystemData& data) override;
};
```

Listing 1: LedModule.h

✓ Config 構造体のポイント

LedConfig にはピン番号だけを持つ。ピン番号を LedModule 内にハードコードせず、Config 構造体で外部から注入する。これにより、同じ LedModule クラスを別プロジェクトでピンだけ変えて再利用できる。

✓ Data 構造体のデフォルト値

LedData の state = false は「updateOutput() が 1 度も呼ばれていない状態でも安全」を保証する。ロジックフェーズが state を true に変更するまで、LED は消灯したままとなる。

2.2 実装（LedModule.cpp）

```cpp
#include "LedModule.h"
#include "SystemData.h"  // ヘッダーでは前方宣言のみ、cppで完全な定義を取得

LedModule::LedModule(const LedConfig& config) : _config(config) {}

bool LedModule::init() {
    pinMode(_config.ledPin, OUTPUT);
    Serial.println("[Led] init OK");
    return true;
}

void LedModule::updateOutput(SystemData& data) {
    digitalWrite(_config.ledPin, data.led.state ? HIGH : LOW);
}
```

Listing 2: LedModule.cpp

✓ 出力モジュールの責務

updateOutput() は**自身の Data 構造体の値に従って動作するだけ**。「いつ点灯するか」「何の条件で消すか」などのロジックは一切書かない。それはロジックフェーズ（applyPattern() 等）の仕事である。

✓ .h/.cpp 分離のルール

ヘッダーでは SystemData.h をインクルードせず、IModule.h 内の前方宣言だけを利用する。SystemData.h のインクルードは.cpp 側で行う。これにより LedModule.h → SystemData → LedModule.h の循環依存を防ぐ。

2.3 ProjectConfig.h での定義

```cpp
const LedConfig LED_CONFIG = {
    .ledPin = 2,  // GPIO2: オンボードLED
};
```

Listing 3: ProjectConfig.h での Config 定義

回路設計者はこのファイルだけ見ればピンアサインが分かる。LED_CONFIG という大文字スネークケースの命名規則も守っている。

3 【基本】ButtonModule — デバウンスパターン

ButtonModule で学べるパターン

- ModuleTimer を使ったデバウンス処理
- エッジ検出（justPressed / justReleased）
- 入力モジュールの基本形（ハードウェアから読んで SystemData に書くだけ）

3.1 Config/Data 構造体

```
struct ButtonConfig {
    uint8_t       pin;          // ボタンピン
    bool          activeLow;    // true: LOW=押下（プルアップ）
    unsigned long debounceMs;   // デバウンス安定時間 [ms]
};

struct ButtonData {
    bool pressed      = false;  // デバウンス後の押下状態
    bool justPressed  = false;  // 今ループで押下された（立ち上がりエッジ）
    bool justReleased = false;  // 今ループで離された（立ち下がりエッジ）
};
```

Listing 4: ButtonModule.h — Config/Data

✓ Config に動作パラメータを含める

debounceMs はピンアサインではないが、モジュールの動作特性を決める設定なので Config に含める。ハードウェア環境によって最適なデバウンス時間は異なるため、外部から注入可能にしておく。

3.2 デバウンスの実装

```
void ButtonModule::updateInput(SystemData& data) {
    // 生の入力値を読み取り
    bool rawPressed = digitalRead(_config.pin);
    if (_config.activeLow) {
        rawPressed = !rawPressed;
    }

    // 入力値が変化したらデバウンスタイマーをリセット
    if (rawPressed != _lastRaw) {
        _debounceTimer.setTime();
        _lastRaw = rawPressed;
    }

    // デバウンス安定時間を超えたら状態を確定
    if (_debounceTimer.getNowTime() >= _config.debounceMs) {
        _stableState = _lastRaw;
    }

    // エッジ検出
```

```
    data.button.pressed      = _stableState;
    data.button.justPressed  = (_stableState && !_prevStable);
    data.button.justReleased = (!_stableState && _prevStable);
    _prevStable = _stableState;
}
```

Listing 5: ButtonModule.cpp — updateInput()

✓ ModuleTimer によるデバウンス

仕様書のデバウンスパターンそのまま。入力が変化するたびに `setTime()` でタイマーをリセットし、`debounceMs` の間安定したら状態を確定する。`delay()` は一切使わず、毎ループ非同期で判定している。

✓ エッジ検出を Data 構造体に含める

`justPressed` と `justReleased` を Data 構造体に含めることで、ロジックフェーズで「ボタンが今押された瞬間」を簡単に検出できる。`if (data.button.justPressed)` と書くだけでよく、ロジック側で前回値を保持する必要がない。

3.3 ロジックフェーズでの使用例

```
void applyPattern(SystemData& data) {
    // ボタンが押された瞬間にLEDをトグル
    if (data.button.justPressed) {
        data.led.state = !data.led.state;
    }

    // ボタン押下中にブザーを鳴らす
    if (data.button.pressed) {
        data.buzzer.frequency = 1000;
        data.buzzer.requestPlay = true;
    }
}
```

Listing 6: ロジックフェーズでの ButtonData 参照

4 【基本】BatteryModule — ADC + 移動平均フィルタ

BatteryModule で学べるパターン

- ModuleTimer による周期サンプリング
- リングバッファを使った移動平均フィルタ
- ADC 値の電圧変換（分圧回路対応）
- isValid フラグによる「データが十分溜まったか」の表現

4.1 Config/Data 構造体

```
static constexpr uint8_t BATTERY_FILTER_SIZE = 16;

struct BatteryConfig {
    uint8_t        adcPin;
    float          voltageDivider;       // 分圧比（例：2.0 = 1/2分圧）
    float          adcRefVoltage;        // ADC基準電圧 [V]
    uint16_t       adcResolution;        // ADC分解能（例：ESP32は4095、AVRは1023）
    unsigned long  sampleIntervalMs;
    float          lowVoltageThreshold;  // 低電圧警告閾値 [V]
};

struct BatteryData {
    float voltage    = 0.0f;   // フィルタ済み電圧 [V]
    float rawVoltage = 0.0f;   // 最新の生電圧 [V]
    bool  isValid    = false;  // サンプルが十分溜まったらtrue
    bool  isLow      = false;  // 低電圧警告フラグ
};
```

Listing 7: BatteryModule.h — Config/Data

✓ Config にハードウェア特性を含める

voltageDivider（分圧比）や adcRefVoltage（基準電圧）はハードウェア構成に依存する値。これらを Config に含めることで、異なる回路構成でも同じモジュールコードを再利用できる。

4.2 リングバッファと移動平均

```
void BatteryModule::updateInput(SystemData& data) {
    if (_sampleTimer.getNowTime() < _config.sampleIntervalMs) {
        return;  // 周期未到達なら何もしない（ノンブロッキング）
    }
    _sampleTimer.setTime();

    // ADC読み取り -> 電圧変換
    uint16_t adcRaw = analogRead(_config.adcPin);
    float voltage = _adcToVoltage(adcRaw);

    // リングバッファに追加
    _samples[_sampleIndex] = voltage;
    _sampleIndex = (_sampleIndex + 1) % BATTERY_FILTER_SIZE;
    if (_sampleCount < BATTERY_FILTER_SIZE) {
        _sampleCount++;
    }

    // SystemDataに書き込み
    data.battery.rawVoltage = voltage;
    data.battery.voltage    = _calcAverage();
    data.battery.isValid    = (_sampleCount >= BATTERY_FILTER_SIZE);
    data.battery.isLow      = (data.battery.voltage < _config.lowVoltageThreshold)
        ;
}
```

Listing 8: BatteryModule.cpp — updateInput()

✓ isValid の使い方

isValid は「移動平均のサンプルが BATTERY_FILTER_SIZE に達したか」を示す。起動直後はサンプルが少なく平均値が不正確なため、ロジックフェーズは isValid == true になるまで電圧値を信用しないようにできる。仕様書の「デフォルト値は正常値と区別できる無効値」ルールと同じ考え方。

✓ 周期サンプリングの定石

updateInput() の冒頭で getNowTime() < interval なら即 return するパターンは、周期実行の定石。ガード節として最初に書くことで、残りのコードのインデントが深くならない。

5 【基本】ServoModule — PWM 出力パターン

ServoModule で学べるパターン

- LEDC（PWM）の初期化と制御
- 「変化がある場合のみ更新」による無駄な出力の抑制
- currentAngle による出力状態のフィードバック

5.1 Config/Data 構造体

```
struct ServoConfig {
    uint8_t  pin;            // サーボ信号ピン
    uint8_t  pwmChannel;     // LEDCチャネル番号（0-15）
    uint16_t minPulseUs;     // 最小パルス幅 [us]（例: 500）
    uint16_t maxPulseUs;     // 最大パルス幅 [us]（例: 2500）
    uint8_t  defaultAngle;   // 初期角度 [度]（0-180）
};

struct ServoData {
    uint8_t targetAngle  = 90;  // 目標角度（ロジックフェーズが設定）
    uint8_t currentAngle = 90;  // 現在の角度（モジュールが更新）
};
```

Listing 9: ServoModule.h — Config/Data

✓ target と current の分離

targetAngle はロジックフェーズが書き込む「指示値」、currentAngle はモジュールが更新する「実際の出力値」。この分離により、ロジックフェーズが「今サーボがどの角度にいるか」を確認できる。

5.2 updateOutput() の実装

```
void ServoModule::updateOutput(SystemData& data) {
    uint8_t target = constrain(data.servo.targetAngle, 0, 180);

    // 変化がある場合のみPWM更新
    if (target != data.servo.currentAngle) {
        ledcWrite(_config.pwmChannel, _angleToDuty(target));
        data.servo.currentAngle = target;
    }
}
```

Listing 10: ServoModule.cpp — updateOutput()

✓ 変化検出パターン

毎ループ PWM を書き込むのではなく、`target != current` の場合のみ更新する。同じ値を毎回書き込んでも動作に問題はないが、不要なハードウェアアクセスを避けるのは良い習慣。特に I2C/SPI 経由のデバイスでは通信コストの削減になる。

✓ constrain による安全性

`constrain()` でサーボの物理的な可動範囲に制限している。ロジックフェーズが誤って範囲外の値を設定しても、モジュール側で安全に丸める。入力バリデーションはモジュールの責務。

6 【基本】BuzzerModule — deinit() パターン

BuzzerModule で学べるパターン

- deinit() による PWM リソースの明示的解放
- リクエスト方式による出力制御（requestPlay フラグ）
- ModuleTimer による時間指定再生の自動停止

6.1 Data 構造体のリクエストパターン

```
struct BuzzerData {
    uint16_t frequency = 0;        // 再生周波数 [Hz]（0=消音）
    unsigned long durationMs = 0; // 再生時間 [ms]（0=無制限）
    bool     requestPlay = false; // ロジックフェーズからの再生リクエスト
};
```

Listing 11: BuzzerModule.h — Data

✓ リクエストフラグパターン

`requestPlay` は「ロジックフェーズからの指示を 1 回だけ処理する」ためのフラグ。`updateOutput()` が処理後に `false` に戻すため、次ループでは再度処理しない。LedModule のように毎ループ同じ状態を出力するのではなく、「イベント的に 1 回だけ何かをする」場合にこのパターンを使う。

6.2 updateOutput() の実装

```
void BuzzerModule::updateOutput(SystemData& data) {
    // 再生リクエスト処理
    if (data.buzzer.requestPlay) {
        data.buzzer.requestPlay = false;  // 処理済みフラグクリア

        if (data.buzzer.frequency > 0) {
            _toneOn(data.buzzer.frequency);
            if (data.buzzer.durationMs > 0) {
                _durationTimer.setTime();
                _timed = true;
            } else {
                _timed = false;  // 無制限再生
            }
        } else {
            _toneOff();
        }
    }

    // 時間指定ありの場合、経過したら自動消音
    if (_playing && _timed) {
        if (_durationTimer.getNowTime() >= data.buzzer.durationMs) {
            _toneOff();
        }
    }
}
```

Listing 12: BuzzerModule.cpp — updateOutput()

6.3 deinit() の実装

```
void BuzzerModule::deinit() {
    _toneOff();
    ledcDetachPin(_config.pin);  // PWMチャネルとピンの紐付けを解除
    Serial.println("[Buzzer] deinit OK");
}
```

Listing 13: BuzzerModule.cpp — deinit()

✓ deinit() の実装が必要なケース

`IModule::deinit()` はデフォルトで空実装。BuzzerModule のように**ハードウェアリソース（PWM チャネル）を占有する**モジュールでは、スリープ前やモジュール停止時にリソースを明示的に解放する必要がある。

`ledcDetachPin()` を呼ばないと、PWM チャネルが占有されたままスリープに入り、復帰後に同じチャネルを使えなくなる可能性がある。

▷ deinit() が不要なモジュール

LedModule や ButtonModule は `pinMode()` で設定した GPIO を使うだけで、特別なリソースを占有しない。こうしたモジュールでは `deinit()` をオーバーライドしなくてよい。

7 【中級】Mpu6500Module — I2C バスパターン

Mpu6500Module で学べるパターン

- 通信バスパターン（コンストラクタでバスポインタを受け取る）
- ヘッダーでのバスクラス前方宣言（`<Wire.h>`依存を.cpp に閉じ込める）
- WHO_AM_I による init() でのデバイス識別
- 周期サンプリング + バースト読み取り

7.1 バスポインタの受け取り

```
// TwoWireはポインタのみ使用するため前方宣言で十分
class TwoWire;

class Mpu6500Module : public IModule {
private:
    Mpu6500Config _config;
    TwoWire*      _wire;        // コンストラクタで受け取るバスポインタ
    ModuleTimer   _sampleTimer;
public:
    explicit Mpu6500Module(const Mpu6500Config& config, TwoWire* wire);
    // ...
};
```

Listing 14: Mpu6500Module.h — バスの前方宣言とコンストラクタ

✓ ヘッダーでの前方宣言

`class TwoWire;` と前方宣言することで、ヘッダーに`<Wire.h>`をインクルードする必要がない。ポインタや参照の宣言には前方宣言だけで十分。これにより、`Mpu6500Module.h` をインクルードするファイルに `Wire.h` の依存が波及しない。

✓ Config にバスポインタを含めない理由

`Mpu6500Config` には I2C アドレスとピン情報だけを含め、`TwoWire*`は含めない。バスポインタを Config に含めると、`ProjectConfig.h` でバスインスタンスの `extern` 宣言が必要になり、インクルード順序の制約が生じる。コンストラクタの別引数にすることで、Config は純粋な設定値のみを持つ。

7.2 main.cpp でのバス管理

```
// バスインスタンス（グローバルスコープで生成）
static TwoWire mpuWire = TwoWire(0);

// モジュール生成（Config + バスポインタ）
Mpu6500Module mpu6500Module(MPU6500_CONFIG, &mpuWire);

void setup() {
    // バス初期化は全モジュールのinit()より前に実行
    mpuWire.begin(MPU6500_CONFIG.sdaPin, MPU6500_CONFIG.sclPin);
    // ...
    mpu6500Module.init();  // init()ではbus.begin()を呼ばない
}
```

Listing 15: main.cpp — バスの生成と初期化

✓ init() で bus.begin() を呼ばない

バスの初期化は `main.cpp` の `setup()` で一元管理する。モジュールの `init()` ではデバイス固有の初期化（WHO_AM_I の確認、レジスタ設定等）のみ行う。同じバスを複数モジュールが共有する場合、各モジュールが `begin()` すると二重初期化になる。

8 【中級】DisplayBoardModule — 入出力統合 + ライブラリ内部管理

DisplayBoardModule で学べるパターン

- 入出力統合モジュール（updateInput + updateOutput の両方をオーバーライド）
- 外部ライブラリ（TFT_eSPI）をモジュール内部にカプセル化する方法
- ModuleTimer による FPS 制御（更新頻度の制限）
- Data 構造体で入力データ（タッチ座標）と出力データ（表示テキスト）を統合管理

8.1 なぜ 1 モジュールに統合するのか

TFT ディスプレイボードは「タッチ入力」と「LCD 出力」の 2 機能を持つが、ハードウェア上は同じ SPI バスを共有する 1 枚の基板であり、同じドライバオブジェクト（TFT_eSPI）で管理される。このような場合、BleModule と同じく **1 つのモジュールで入出力の両方をオーバーライド**する方式が適切である。

✓ 入出力統合の判断基準

「入力と出力を別モジュールに分けるか、1 モジュールに統合するか」は、**ハードウェアの単位**で判断する。同一基板・同一ドライバで管理されるデバイスは統合し、物理的に独立したデバイスは別モジュールにする。BleModule の送受信統合と同じ考え方。

8.2 ライブラリの内部管理パターン

```
class TFT_eSPI;  // ヘッダへの依存を防ぐ

class DisplayBoardModule : public IModule {
private:
    DisplayBoardConfig _config;
    TFT_eSPI*          _tft;    // 内部で生成・管理
public:
    DisplayBoardModule(const DisplayBoardConfig& config);
    ~DisplayBoardModule();
    bool init()                          override;
    void updateInput(SystemData& data)   override;
    void updateOutput(SystemData& data)  override;
    void deinit()                        override;
};
```

Listing 16: DisplayBoardModule.h — TFT_eSPI の前方宣言

```
#include <TFT_eSPI.h>

DisplayBoardModule::DisplayBoardModule(const DisplayBoardConfig& config)
    : _config(config), _tft(new TFT_eSPI()) {}

DisplayBoardModule::~DisplayBoardModule() {
    delete _tft;
}
```

Listing 17: DisplayBoardModule.cpp — コンストラクタで TFT_eSPI を生成

✓ 外部バスパターンとの違い

Mpu6500Module は `main.cpp` で I2C バスを生成し、コンストラクタ引数で渡した（外部バスパターン）。DisplayBoardModule では TFT_eSPI が内部で SPI バスを管理するため、外部から渡す必要がない。SPI ピン設定は `platformio.ini` の build_flags でコンパイル時に決定される。

！既存ライブラリの制約

TFT_eSPI や LovyanGFX のように、ライブラリ内部でバスインスタンスを保持するライブラリでは、アーキテクチャの「外部からバスポインタを渡す」パターンをそのまま適用できない。この場合、ライブラリごとモジュール内部にカプセル化し、Config 構造体や build_flags で設定を渡す方式で対応する。

8.3 Data 構造体の入出力統合

```
struct DisplayBoardData {
    // 出力: 表示テキスト（ロジックフェーズが書き込む）
    char line1[48] = "";
    char line2[48] = "";
    char line3[48] = "";
    char line4[48] = "";
    char line5[48] = "";        // 黄色テキスト
    // 入力: タッチ状態（入力フェーズで更新）
```

```
    bool      touchPressed = false;
    uint16_t touchX        = 0; // ピクセル座標
    uint16_t touchY        = 0;
};
```

Listing 18: DisplayBoardModule.h — Data

✓ 入力と出力のデータを 1 つの Data 構造体に

DisplayBoardData はタッチ入力（touchPressed 等）と表示出力（line1 等）の両方を含む。入力フェーズでタッチ座標が書き込まれ、ロジックフェーズでそれを参照して表示テキストを設定し、出力フェーズで画面に描画する。BleModule の BleData（受信データ + 送信データ）と同じ構成。

8.4 ロジックフェーズでの表示内容設定

```
void applyPattern(SystemData& data) {
    // MPU-6500データを表示用にフォーマット
    if (data.mpu.isValid) {
        snprintf(data.display.line1, sizeof(data.display.line1),
                 "Ax:%.2f Ay:%.2f Az:%.2f",
                 data.mpu.accelX, data.mpu.accelY, data.mpu.accelZ);
    }
    // タッチ座標を表示用にフォーマット
    if (data.display.touchPressed) {
        snprintf(data.display.line5, sizeof(data.display.line5),
                 "Touch: %3d, %3d",
                 data.display.touchX, data.display.touchY);
    }
}
```

Listing 19: main.cpp — applyPattern() での表示設定

✓ 3 フェーズモデルのデータフロー

入力フェーズで DisplayBoardModule が data.display.touchPressed/X/Y に値を書き込み、ロジックフェーズで MPU データやタッチ座標を文字列に変換して data.display.line1 等に書き込み、出力フェーズで DisplayBoardModule が画面に描画する。入力→ロジック→出力の典型的なデータフローの実例。

9 【中級】CameraModule — PSRAM バッファ + フレーム管理

CameraModule で学べるパターン

- PSRAM への大容量バッファ配置
- フレームバッファのライフサイクル管理（取得→使用→解放）
- Data 構造体にメタデータのみ格納し、生データはモジュール内に保持
- 多数のピンを Config 構造体で一元管理

9.1 Config の大量ピン管理

```
struct CameraConfig {
    int8_t pwdnPin, resetPin;           // 電源/リセット
    int8_t xclkPin, siodPin, siocPin;   // クロック/SCCB
    int8_t d0Pin, d1Pin, d2Pin, d3Pin;  // DVPデータバス
    int8_t d4Pin, d5Pin, d6Pin, d7Pin;
    int8_t vsyncPin, hrefPin, pclkPin;  // 同期
    uint32_t xclkFreqHz;
    int frameSize, pixFormat;
    int jpegQuality;
    int fbCount;
};
```

Listing 20: CameraModule.h — Config（多数のピン）

✓ Config 構造体の威力

カメラモジュールはピンが 16 本以上ある。コンストラクタの引数で 16 個のピンを渡すと順序を間違える危険があるが、Config 構造体なら`.d0Pin = 11` のように名前で指定できる。Config 構造体のピンアサインが `ProjectConfig.h` に集約されるため、回路設計者はこのファイルだけでピンの確認・変更ができる。

9.2 フレームバッファの管理

```
// updateInput()はメタデータだけをSystemDataに書き込む
void CameraModule::updateInput(SystemData& data) {
    if (_prevFb) {
        esp_camera_fb_return(_prevFb);  // 前回のフレームを自動解放
        _prevFb = nullptr;
    }
    camera_fb_t* fb = esp_camera_fb_get();
    if (fb) {
        data.camera.frameReady = true;
        data.camera.width = fb->width;
        data.camera.height = fb->height;
        data.camera.frameSize = fb->len;
        _prevFb = fb;  // モジュール内で保持
    }
}

// ロジックフェーズから生データにアクセスする場合
camera_fb_t* CameraModule::getFrameBuffer() { return _prevFb; }
void CameraModule::releaseFrame() {
    if (_prevFb) {
        esp_camera_fb_return(_prevFb);
        _prevFb = nullptr;
    }
}
```

Listing 21: CameraModule — フレームバッファ管理

✓ Data 構造体に生データを入れない

camera_fb_t*（フレームバッファポインタ）は SystemData に格納しない。SystemData に入れると、他モジュールがフレームの解放タイミングを制御できてしまう。代わに getFrameBuffer() / releaseFrame() でアクセスを制御し、CameraData にはメタデータ（サイズ、解像度等）のみを格納する。

▷ PSRAM の自動配置

fbCount >= 2 の場合、esp_camera ライブラリが自動的に PSRAM にバッファを配置する。Config 構造体の fbCount で制御でき、モジュール内部で PSRAM 判定のコードを書く必要はない。

10 【応用】DriveMotorModule — PWM 2 ピン制御 + 統合部品

DriveMotorModule で学べるパターン

- H ブリッジによる方向制御（IN1/IN2 + PWM）
- 逆転保護（ブレーキ自動挿入）
- drive() メソッドによる SystemData 非経由の直接制御
- 統合モジュールの「内部部品」としての使い方

10.1 2 つの使い方

DriveMotorModule は 2 つのモードで使える。

1. **単体使用** — outputModules[] に登録し、SystemData 経由で制御
2. **統合モジュールの内部部品** — ChassisModule 等が内部で drive() を呼ぶ

```
class DriveMotorModule : public IModule {
public:
    // SystemData経由の制御（単体使用時）
    void updateOutput(SystemData& data) override;

    // SystemData非経由の直接制御（統合モジュールから使用）
    void drive(float power);
    void brake();
};
```

Listing 22: DriveMotorModule.h — drive() メソッド

✓ updateOutput() と drive() の使い分け

updateOutput() は SystemData の DriveMotorData.power を読んで drive() を呼ぶラッパー。統合モジュールの内部部品として使う場合は drive() を直接呼び、updateOutput() は空実装（呼ばれない）となる。この設計により、同じモジュールコードを単体でも統合でも使い回せる。

10.2 逆転保護

```
void DriveMotorModule::drive(float power) {
    Direction newDir = (power > 0) ? FORWARD
                     : (power < 0) ? REVERSE : STOP;

    // 正転<->逆転の直接切り替え時、1フレーム分のブレーキを挿入
    if ((_lastDir == FORWARD && newDir == REVERSE) ||
        (_lastDir == REVERSE && newDir == FORWARD)) {
        brake();
        _lastDir = STOP;
        return;  // 次のupdateOutput()で実際の方向転換が行われる
    }
    // ... 実際のPWM制御 ...
}
```

Listing 23: DriveMotorModule.cpp — 逆転保護

✓ 1 フレーム遅延によるモーター保護

正転→逆転の直接切り替えはモーターやドライバ IC に大きな負荷がかかる。1 フレーム分（1 ループ分）のブレーキを挿入することで、物理的に安全な方向転換を実現する。3 フェーズモデルの「毎ループ 1 回だけ呼ばれる」性質を利用した保護パターン。

11 【応用】ChassisModule — 機能統合パターン

ChassisModule で学べるパターン

- 複数モジュールを内部に保持する「機能統合系モジュール」
- 内部モジュールの init()/updateOutput()/deinit() を親が管理
- ベクトル合成による高レベル制御（3 軸入力→ 4 輪出力）
- 内部モジュールの SystemData 登録が不要なケース

11.1 内部モジュールの保持

```
struct ChassisConfig {
    DriveMotorConfig motors[4];     // 4輪分のモーターConfig
    WheelPattern     wheelPattern;
    float            maxOutputPercent;
};

class ChassisModule : public IModule {
private:
    ChassisConfig      _config;
    DriveMotorModule   _motors[4];  // 内部にモジュールを直接保持
public:
    ChassisModule(const ChassisConfig& config);
    bool init() override;
    void updateOutput(SystemData& data) override;
```

```
    void deinit() override;
};
```

Listing 24: ChassisModule.h — 内部に DriveMotorModule を 4 つ保持

✓ 内部モジュールは IModule 配列に登録しない

`_motors[4]` は `ChassisModule` のメンバ変数であり、`main.cpp` の `outputModules[]` には登録しない。`SystemData` にも `DriveMotorData` を含める必要がない。外部からは `ChassisModule` 1 つだけが見え、3 軸入力だけで 4 輪を制御できる。

11.2 親モジュールからの制御委譲

```
ChassisModule::ChassisModule(const ChassisConfig& config)
    : _config(config),
      _motors{
          DriveMotorModule(config.motors[0]),
          DriveMotorModule(config.motors[1]),
          DriveMotorModule(config.motors[2]),
          DriveMotorModule(config.motors[3]),
      } {}
```

Listing 25: ChassisModule.cpp — init()/deinit() で内部モジュールを管理

▷ コンストラクタの初期化リスト — 初学者向け解説

コロン（:）の後に書かれている部分が「初期化リスト」です。ここでは以下の処理を行っています。

1. `_config(config)` — Config 構造体をコピー保存
2. `_motors{...}` — DriveMotorModule の配列を、各モーターの Config で初期化

配列の初期化は波括弧 {} で囲みます。中の `DriveMotorModule(config.motors[0])` は「config.motors[0] の設定で DriveMotorModule を作る」という意味です。

これは「コンストラクタの中で 4 つの子モジュールを一気に作る」処理です。初期化リスト構文に慣れないうちは「コンストラクタ引数を受け取って、メンバ変数に渡している」とだけ理解すれば十分です。

```

bool ChassisModule::init() {
    for (int i = 0; i < 4; i++) {
        if (!_motors[i].init()) return false;
    }
    return true;
}

void ChassisModule::deinit() {
    for (int i = 0; i < 4; i++) {
        _motors[i].deinit();
    }
}
```

Listing 26: ChassisModule.cpp — init()/deinit()（続き）

```
void ChassisModule::updateOutput(SystemData& data) {
    float outputs[4];
    // ベクトル合成（ホイールパターンに応じた方向行列）
    calcWheelOutputs(data.chassis, outputs);
    // リミッタ + スケーリング
    applyLimiter(outputs, _config.maxOutputPercent);
    // 各内部モジュールの drive() を直接呼ぶ
    for (int i = 0; i < 4; i++) {
        _motors[i].drive(outputs[i]);
        data.chassis.motorOutputs[i] = outputs[i];
    }
}
```

Listing 27: ChassisModule.cpp — updateOutput() でベクトル合成→各モーターを drive()

✓ 機能統合の原則

仕様書の「モジュール間の依存関係は存在しないものとして設計する」ルールに従い、依存関係があるモジュール同士は 1 つに統合する。ChassisModule と DriveMotorModule は明確に依存関係があるため、ChassisModule が DriveMotorModule を内部部品として包含している。

▷ Config 構造体のネスト

`ChassisConfig.motors[4]` は `DriveMotorConfig` の配列。Config 構造体は別の Config 構造体をメンバとして持てる。これにより `ProjectConfig.h` で 4 輪分のピンアサインを一括定義できる。

12 【応用】BleModule — スレッドセーフティパターン

BleModule で学べるパターン

- volatile フラグ方式による安全なデータ受け渡し
- BLE コールバック（別タスク）→内部バッファ→ updateInput() → SystemData
- deinit() による BLE リソースの解放
- 双方向通信（RX: updateInput() で受信反映、TX: updateOutput() で送信）

12.1 内部バッファと volatile フラグ

```
class BleModule : public IModule {
private:
    // 内部バッファ（BLE コールバックスレッドが書き込む）
    uint8_t       _rxBuffer[BLE_RX_BUFFER_SIZE] = {};
    uint8_t       _rxBufferLength = 0;
    volatile bool _dataReceived   = false;  // volatile フラグ
    volatile bool _clientConnected = false;
public:
    // BLE コールバックから呼ばれる（別タスクで実行される）
    void onReceive(const uint8_t* payload, uint8_t length);
    // ...
```

```
};
```

Listing 28: BleModule.h — 内部バッファ

！ なぜ volatile が必要か

BLE コールバックは Arduino のメインループとは別の FreeRTOS タスクで実行される。volatile がないと、コンパイラが_dataReceived の値をレジスタにキャッシュし、メインループ側で変更を検出できない場合がある。volatile は「この変数は外部から変更されるかもしれない」とコンパイラに伝えるキーワード。

12.2 コールバック→内部バッファ→ SystemData

```
// BLEコールバック（別タスクから呼ばれる）
void BleModule::onReceive(const uint8_t* payload, uint8_t length) {
    memcpy(_rxBuffer, payload, length);  // (1) バッファに書き込み
    _rxBufferLength = length;
    _dataReceived = true;                // (2) フラグを立てる
}

// updateInput()は入力フェーズで呼ばれる（メインタスク）
void BleModule::updateInput(SystemData& data) {
    data.ble.connected = _clientConnected;

    if (_dataReceived) {                              // (3) フラグ確認
        memcpy(data.ble.rxData, _rxBuffer, _rxBufferLength); // (4) コピー
        data.ble.rxLength  = _rxBufferLength;
        data.ble.newDataIn = true;
        _dataReceived = false;                        // (5) フラグクリア
    }
}

// updateOutput()は出力フェーズで呼ばれる
void BleModule::updateOutput(SystemData& data) {
    // 送信リクエスト処理
    if (data.ble.sendRequest && _clientConnected && _txChar) {
        _txChar->setValue(data.ble.txData, data.ble.txLength);
        _txChar->notify();
        data.ble.sendRequest = false;
    }
}
```

Listing 29: BleModule.cpp — データ受け渡しの流れ

✓ volatile フラグ方式の安全性

bool 型の読み書きは原子的（1 命令で完了する）。「バッファに全データを書き込んだ後にフラグを立てる」→「フラグを確認してからバッファを読む」の順序により、バッファの途中状態を読むことがない。mutex やセマフォは不要で、割り込みコンテキストでも安全に使える。

12.3 BLE コールバックの登録

```
// BLEライブラリのコールバッククラスを継承
class BleRxCallbacks : public BLECharacteristicCallbacks {
private:
    BleModule* _module;
public:
    BleRxCallbacks(BleModule* module) : _module(module) {}
    void onWrite(BLECharacteristic* characteristic) override {
        std::string value = characteristic->getValue();
        uint8_t len = min((int)value.length(), BLE_RX_BUFFER_SIZE);
        _module->onReceive(
            reinterpret_cast<const uint8_t*>(value.data()), len);
    }
};
```

Listing 30: BleModule.cpp — コールバッククラス

✓ コールバックからモジュールへの橋渡し

BLE ライブラリはコールバッククラスの継承を要求するため、内部クラスでラップして `BleModule::onReceive()` に橋渡しする。コールバッククラスはモジュールのポインタを保持し、受信データをモジュールの内部バッファに渡す。

12.4 送信の流れ（ロジックフェーズ→ updateOutput()）

```
void applyPattern(SystemData& data) {
    // BLE受信データの処理
    if (data.ble.newDataIn) {
        // 受信データを処理...
        data.ble.newDataIn = false;  // 処理済みフラグクリア
    }

    // BLE送信（応答を返す例）
    if (needToRespond) {
        memcpy(data.ble.txData, response, responseLen);
        data.ble.txLength = responseLen;
        data.ble.sendRequest = true;  // 次のupdateOutput()で送信される
    }
}
```

Listing 31: ロジックフェーズでの送信リクエスト

▷ 双方向通信のデータフロー

受信: BLE コールバック → 内部バッファ → updateInput() → SystemData.ble.rxData → ロジックフェーズ

送信: ロジックフェーズ → SystemData.ble.txData → updateOutput() → BLE Notify

BleModule は `inputModules[]` と `outputModules[]` の両方に登録する。受信処理は入力フェーズの `updateInput()` で、送信処理は出力フェーズの `updateOutput()` で実行される。

13 【応用】WifiModule — ステートマシンパターン

WifiModule で学べるパターン

- enum class による状態定義と switch 文による状態遷移
- ModuleTimer によるタイムアウト・再接続間隔の管理
- リクエストフラグによる外部からの制御（接続/切断指示）
- deinit() による WiFi リソースの解放

13.1 状態定義

```
enum class WifiState : uint8_t {
    DISCONNECTED,  // 未接続（接続リクエスト待ち）
    CONNECTING,    // 接続試行中
    CONNECTED,     // 接続済み
    RECONNECTING,  // 再接続試行中
    FAILED,        // 接続失敗（リトライ上限超過）
};
```

Listing 32: WifiModule.h — 状態定義

✓ Data 構造体にステートを公開

`WifiData.state` として enum 値を SystemData に公開することで、ロジックフェーズが現在の接続状態に応じた判断を行える。`isConnected` は簡易参照用の bool で、`state == CONNECTED` の省略形。

13.2 ステートマシンの実装

```
void WifiModule::updateInput(SystemData& data) {
    // ロジックフェーズからのリクエスト処理
    if (data.wifi.requestConnect) {
        data.wifi.requestConnect = false;
        if (_state == WifiState::DISCONNECTED
            || _state == WifiState::FAILED) {
            _retryCount = 0;
            _startConnect();
        }
    }

    // ステートマシン
    switch (_state) {
        case WifiState::CONNECTING:
        case WifiState::RECONNECTING:
            if (WiFi.status() == WL_CONNECTED) {
                _state = WifiState::CONNECTED;
                _retryCount = 0;
            } else if (_stateTimer.getNowTime()
                        >= _config.connectTimeoutMs) {
                WiFi.disconnect();
                _retryCount++;
```

```
                if (_config.maxRetries > 0
                    && _retryCount >= _config.maxRetries) {
                    _state = WifiState::FAILED;
                } else {
                    _state = WifiState::RECONNECTING;
                    _stateTimer.setTime();
                }
            }
            break;

        case WifiState::CONNECTED:
            if (WiFi.status() != WL_CONNECTED) {
                _state = WifiState::RECONNECTING;
                _stateTimer.setTime();
            }
            break;
        // ...
    }

    // SystemDataに状態を反映
    data.wifi.state       = _state;
    data.wifi.isConnected = (_state == WifiState::CONNECTED);
    data.wifi.retryCount  = _retryCount;
}
```

Listing 33: WifiModule.cpp — updateInput() のステートマシン

✓ ノンブロッキングなステートマシン

WiFi 接続は通常数秒かかるが、`delay()` で待たない。`CONNECTING` 状態では毎ループ `WiFi.status()` を確認し、接続完了かタイムアウトかを判定する。これにより、WiFi 接続中も他のモジュール（LED、センサー等）は正常に動作し続ける。

✓ リクエストフラグパターン

`requestConnect` はロジックフェーズからの「接続して」という指示。BuzzerModule の `requestPlay` と同じパターンで、`updateInput()` が処理後に `false` に戻す。

13.3 ロジックフェーズでの使用例

```
void applyPattern(SystemData& data) {
    // ボタンが押されたらWiFi接続開始
    if (data.button.justPressed
        && !data.wifi.isConnected) {
        data.wifi.requestConnect = true;
    }

    // WiFi接続状態に応じた表示
    switch (data.wifi.state) {
        case WifiState::CONNECTING:
            snprintf(data.display.line1, 48, "WiFi: Connecting...");
            break;
```

```
        case WifiState::CONNECTED:
            snprintf(data.display.line1, 48,
                     "WiFi: OK (RSSI=%d)", data.wifi.rssi);
            break;
        case WifiState::FAILED:
            snprintf(data.display.line1, 48,
                     "WiFi: Failed (%d retries)",
                     data.wifi.retryCount);
            break;
        default: break;
    }
}
```

Listing 34: ロジックフェーズでの WiFi 制御

14 【応用】EncoderModule — 割り込み連携パターン

EncoderModule で学べるパターン

- ハードウェア割り込み（ISR）と volatile 変数の連携
- IRAM_ATTR による割り込みハンドラの RAM 配置
- ISR → volatile 変数→ updateInput() → SystemData のデータフロー
- deinit() による割り込みの解除

14.1 割り込みハンドラの設計

```
class EncoderModule : public IModule {
private:
    EncoderConfig _config;
    ModuleTimer   _sampleTimer;

    // ISRが更新する変数（volatile必須）
    volatile int32_t _isrCount = 0;

    // updateInput()内で使う前回値
    int32_t _prevCount = 0;

    // ISRはstatic関数でなければattachInterruptに渡せない
    static void IRAM_ATTR _isrHandlerA();
    static EncoderModule* _instance;

    void IRAM_ATTR _onPulseA();
    // ...
};
```

Listing 35: EncoderModule.h — 割り込み関連

▷ なぜ static が必要なのか？ — 初学者向け解説

C++ のクラスのメンバ関数（例: `_onPulseA()`）は、内部的に「どのオブジェクトの関数か」という情報（this ポインタ）を持っています。しかし、`attachInterrupt()` が受け付けるのは C 言語形式の「普通の関数ポインタ」だけで、this ポインタを渡す仕組みがありません。
そこで以下の 2 段構えで解決します。

1. **static 関数**を用意する — static 関数は this ポインタを必要としないので、割り込みに登録できる
2. **static なインスタンスポインタ**を用意する — static 関数の中から「どのオブジェクトか」を知るための橋渡し

```
// 1. staticポインタ: 「どのエンコーダか」を覚えておく
static EncoderModule* _instance;

// 2. static関数: 割り込みに登録できる
static void _isrHandlerA() {
    _instance->_onPulseA();  // ここで実際の処理に橋渡し
}

// init()で登録
_instance = this;
attachInterrupt(pin, _isrHandlerA, RISING);
```

これは「C++ のオブジェクト指向と C 言語の割り込みの橋渡し」パターンです。割り込みを使わない通常のモジュールでは不要なので、初めて見る場合はこのパターンだけ覚えれば十分です。

！ ISR（割り込みサービスルーチン）の制約

- `attachInterrupt()` に渡す関数は **static 関数**でなければならない
- ISR 内では `Serial.print()` やメモリ確保等の重い処理は禁止
- ESP32 等では `IRAM_ATTR` を付けて RAM に配置する必要がある（ボード依存）
- ISR から参照する変数は `volatile` を付ける

14.2 ISR → volatile → updateInput() の流れ

```
// ISRハンドラ（IRAM_ATTR = RAM配置、高速アクセス）
void IRAM_ATTR EncoderModule::_isrHandlerA() {
    if (_instance) {
        _instance->_onPulseA();
    }
}

void IRAM_ATTR EncoderModule::_onPulseA() {
    // A相の立ち上がり時にB相を読み取って方向を判定
    if (digitalRead(_config.pinB) == HIGH) {
        _isrCount++;    // CW
    } else {
        _isrCount--;    // CCW
    }
}

```

```
// updateInput()は入力フェーズで呼ばれる
void EncoderModule::updateInput(SystemData& data) {
    // volatile変数を一度だけ読み取り
    int32_t currentCount = _isrCount;

    data.encoder.count = currentCount;

    // 速度計算（一定間隔で実行）
    if (_sampleTimer.getNowTime() >= _config.sampleIntervalMs) {
        unsigned long elapsed = _sampleTimer.getNowTime();
        _sampleTimer.setTime();

        int32_t delta = currentCount - _prevCount;
        _prevCount = currentCount;

        float revolutions = (float)delta / _config.pulsesPerRev;
        float seconds = (float)elapsed / 1000.0f;
        data.encoder.rpm = (seconds > 0)
            ? (revolutions / seconds * 60.0f) : 0.0f;
        data.encoder.isValid = true;
    }
}
```

Listing 36: EncoderModule.cpp — ISR と update()

✓ volatile 変数を一度だけ読み取る

`int32_t currentCount = _isrCount;` で値をローカル変数にコピーしてから使う。volatile 変数を関数内で複数回参照すると、参照のたびに値が変わる可能性がある。一度だけ読み取ることで、同一 updateInput() 内での値の一貫性を保証する。

✓ BleModule との共通パターン

BleModule（コールバック）と EncoderModule（ISR）は、「メインタスク外からのデータ受け渡し」という同じ問題を解決している。どちらも **volatile 変数（またはフラグ）**を介して安全にデータを渡す。BLE はバッファ + フラグ、エンコーダはカウンタ変数という違いはあるが、原理は同じ。

14.3 deinit() で割り込みを解除

```
void EncoderModule::deinit() {
    detachInterrupt(digitalPinToInterrupt(_config.pinA));
    _instance = nullptr;
    Serial.println("[Encoder] deinit OK");
}
```

Listing 37: EncoderModule.cpp — deinit()

✓ 割り込み解除の重要性

割り込みを解除しないままスリープに入ると、スリープ中にも割り込みが発生してウェイクアップの原因になる可能性がある。また、`_instance = nullptr` にすることで、deinit 後に ISR が呼ばれても null チェックで安全にスキップされる。

▷ 複数インスタンスへの拡張

現在の実装は static ポインタ 1 つ（`_instance`）のため、エンコーダモジュールは 1 つしか使えない。複数使う場合は `static EncoderModule* _instances[MAX]` の配列に拡張し、チャネル ID でインデックスする方式に変更する。